

NANYANG
TECHNOLOGICAL
UNIVERSITY

CZ2006/CE2006

Software Engineering

Lecture 16-17

Equivalence Class Testing

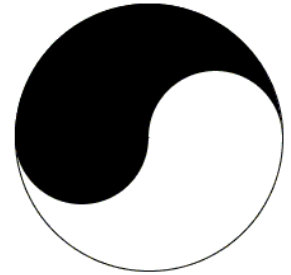
When I launch my app after several hours of development

Unexpected things happen!



A **software bug** is an error, flaw, failure or fault in a computer program or system that causes it to produce an incorrect or unexpected result, or to behave in unintended ways.

Types Of Testing



► **Black Box Testing**

- What to analyze: Requirements and specifications
 - E.g. *“The income of an applicant must be between \$1500 per month and \$8000 per month”*
- Type of black box testing
 - **Equivalence class testing and boundary value testing**
 - State-based testing

► **White Box Testing**

- What to analyze: Implementation details, internal paths and structure
- Type of white box testing
 - **Control flow testing**
 - Data flow testing

Levels Of Testing

- ▶ Unit testing (we learn in this course)
 - ▶ Test an unit **individually**
- ▶ Integration testing
 - ▶ Test units in **combination**
- ▶ System testing
 - ▶ Test **everything** (functionality, usability, reliability, performance, portability, security ...)
- ▶ Acceptance testing (UAT User Acceptance Testing)
 - ▶ **Customer** accepts the software and give us their **money**

Ingredients Of Test Case

Ingredient	Description
Name	Distinct name of test case
Location	Path to test program and its inputs
Input	Input data or commands
Oracle	Expected test output
Log	Actual output produced by the test

Black Box Testing Process

► The tester

1. Analyzes **requirements** or **specifications**
 - equivalence classes and boundary values
2. Chooses **valid** inputs and **invalid** inputs
3. Determines expected outputs (oracle) for chosen inputs
4. Constructs tests with the chosen inputs (e.g., write JUnit program)

The most difficult steps

► The testing engine (e.g., JUnit)

1. Executes the tests
2. Compares actual outputs (log) with the expected outputs (oracle)
3. Determines whether the SUT (System Under Test) functions properly

Black Box Testing Pros and Cons

▶ **Applicability**

- ▶ All levels of system development (Unit, Integration, System, Acceptance)

▶ **Cons**

- ▶ Cannot know how much of the implementation have been tested
- ▶ No notion of testing coverage like in white box testing

▶ **Pros**

- ▶ Directs the tester to a very small subset of test inputs that can highly likely find the bugs

We do Black Box Testing Often...



<https://www.dpreview.com/galleries/6998361880/photos/1163297>

Assumption #1

- ▶ **Verifiable requirements** and API specification exists
 - ▶ Make software more testable from its inception!
- ▶ A requirement like this is **NOT** verifiable!
 - ▶ The company must not hire **too-young** or **too-old** people
 - ▶ The company can hire **juniors** on part-time basis
 - ▶ The company can hire **adult** as full-time employees

Compare this with verifiable requirements on Hire or Not slide!

Assumption #2

- ▶ **Code must be testable!**
- ▶ Examples that make black-box testing less effective

```
if (age==0) hire=NO;  
if (age==1) hire=NO;  
...  
if (age==15) hire=NO;  
if (age==16) hire=PART;  
if (age==17) hire=PART;  
if (age==18) hire=FULL;  
...  
if (age==54) hire=FULL;  
if (age==55) hire=NO;  
if (age==56) hire=NO;  
...  
if (age==98) hire=NO;  
if (age==99) hire=NO;
```

```
if (age>=0 && age <= 15) hire=NO;  
if (age>=16 && age <= 18) hire=PART;  
if (age>=18 && age <= 34) hire=FULL;
```

```
if (age==35 && name=="xingze")  
hire=CEO;
```

```
if (age==35 && name!="xingzc")  
hire=FULL;
```

```
if (age>=36 && age<=54) hire=FULL;  
if (age>=55 && age<=99) hire=NO;
```

Motivating Example: Hire or Not?

- ▶ Test a module implementing the following hiring requirements
 - ▶ 0 – 15, 55 – 99 Do not hire
 - ▶ 16 – 17 Part-time
 - ▶ 18 – 54 Full-time

~~Test ..., -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42,
43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53,
54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64,
65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75,
76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86,
87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97,
98, 99, 100, 101, 102, ...?~~

*Test only one value for
each range, one invalid
value below 0, and one
invalid value above 99*

Equivalence Class Testing – Partition Input Values into Equivalence Classes

- ▶ A set of input values forms an **equivalence class (EC)** if
 - ▶ They all are supposed to produce the **same output**
 - ▶ If one value catches a bug, the others probably will too
 - ▶ If one value does not catch a bug, the others probably will not either

A metaphor



<http://web.1.c2.audiovideoweb.com/1c2web3536/lowcalrice.jpg>

Equivalence Class Testing Process

► The tester

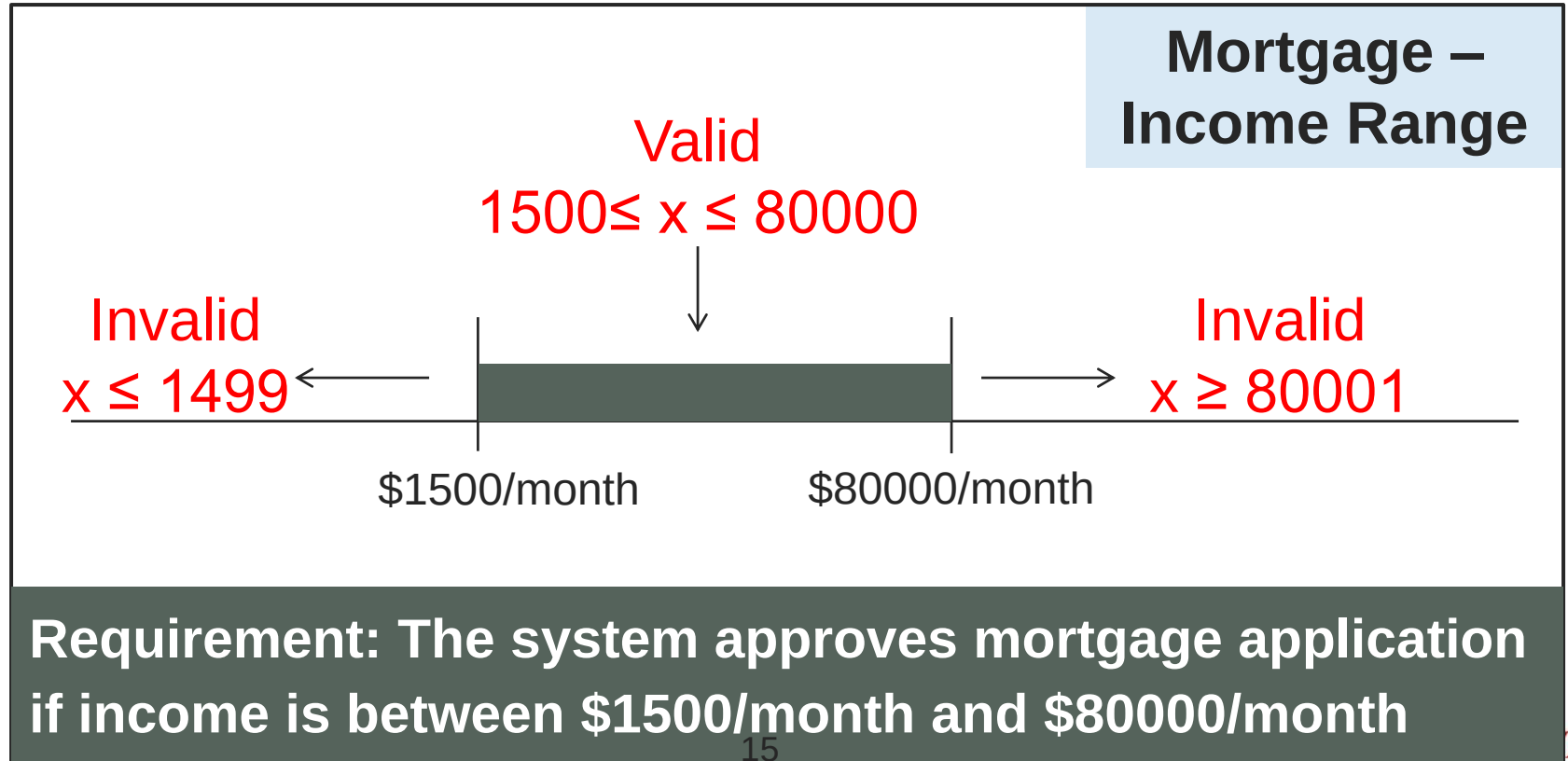
1. Identify the **valid and invalid equivalence classes** that partition the input values
2. Choose **at least one input value** for each equivalence class of each input parameter
3. Determines expected outputs (oracle) for chosen inputs
4. Constructs tests with the chosen inputs (e.g., write JUnit program)

► The testing engine (e.g., JUnit)

1. Executes the tests
2. Compares actual outputs (log) with the expected outputs (oracle)
3. Determines whether the SUT (System Under Test) functions properly

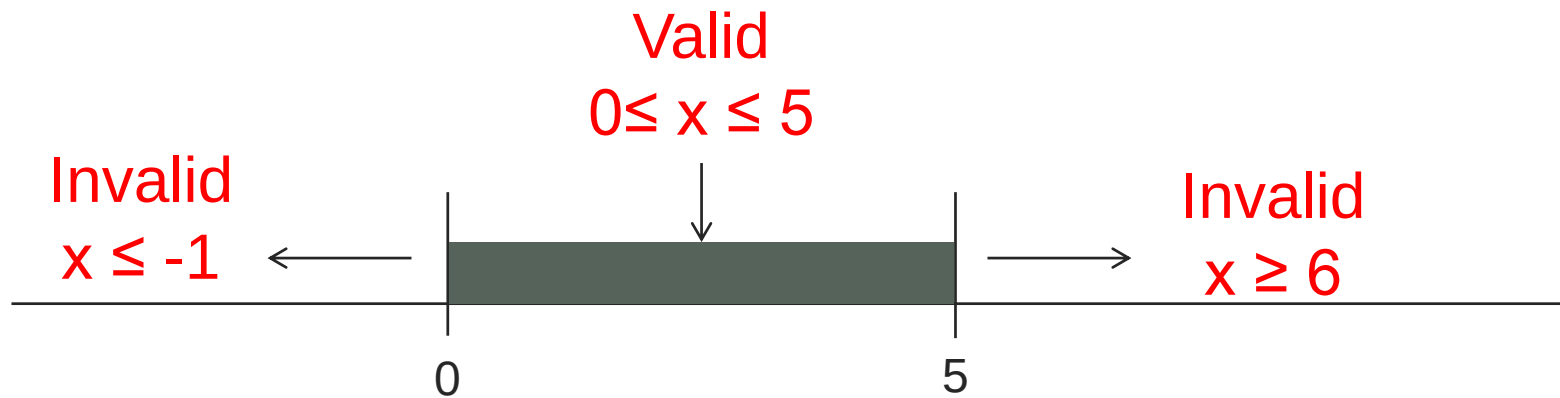
Range of Values

- ▶ **ONE** equivalence class of **VALID** values
- ▶ **TWO** equivalence classes of **INVALID** values



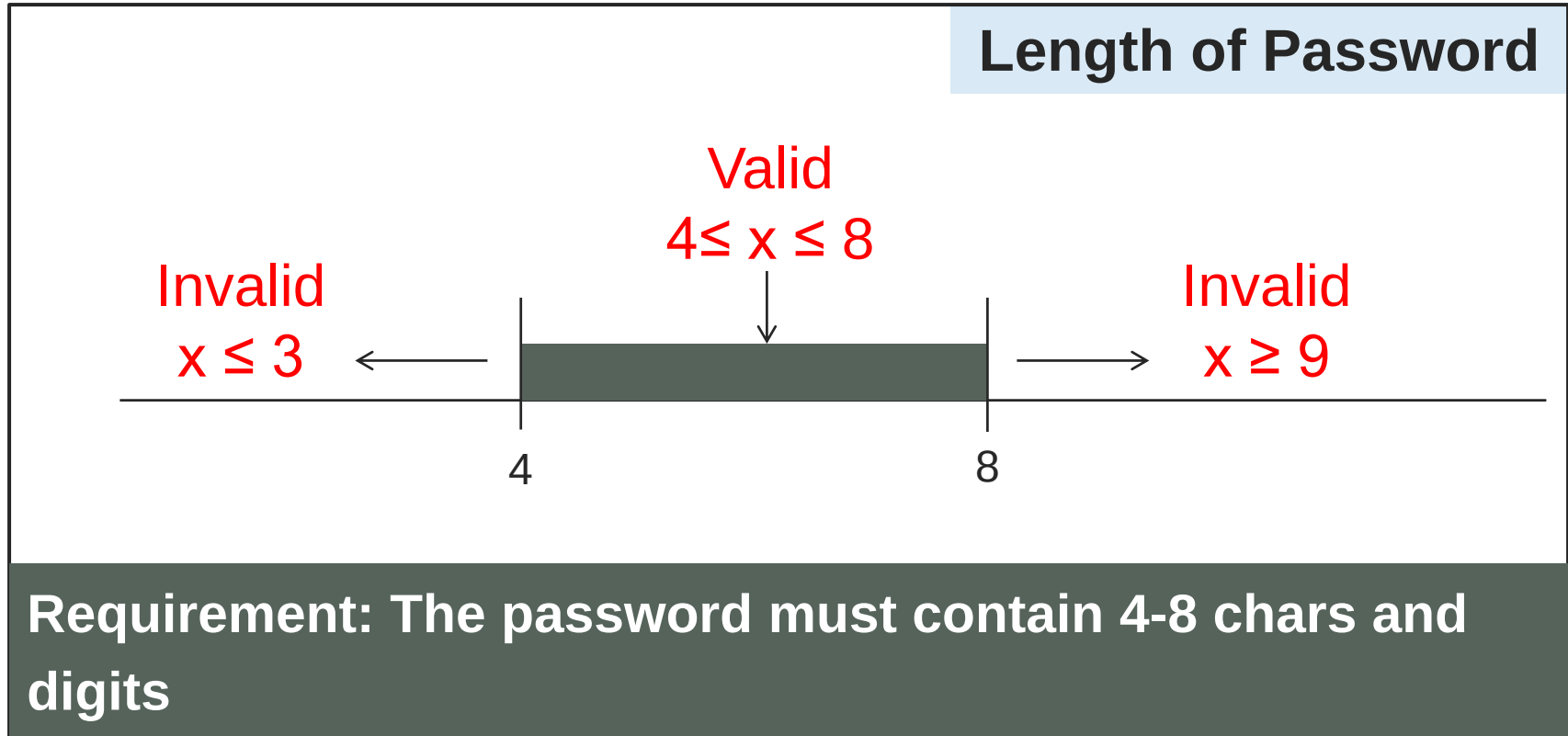
Range of Values – Exercise

**Mortgage –
Number of Mortgages**



Requirement: The system approves application if #mortgage per applicant is between 0 and 5

Range of Values – Exercise



Range of Values – Exercise

Hiring Rules

Valid equivalence classes

$0 \leq \text{age} \leq 15$ (do not hire)
 $16 \leq \text{age} \leq 17$ (hire part-time)
 $18 \leq \text{age} \leq 54$ (hire full-time)
 $55 \leq \text{age} \leq 99$ (do not hire)

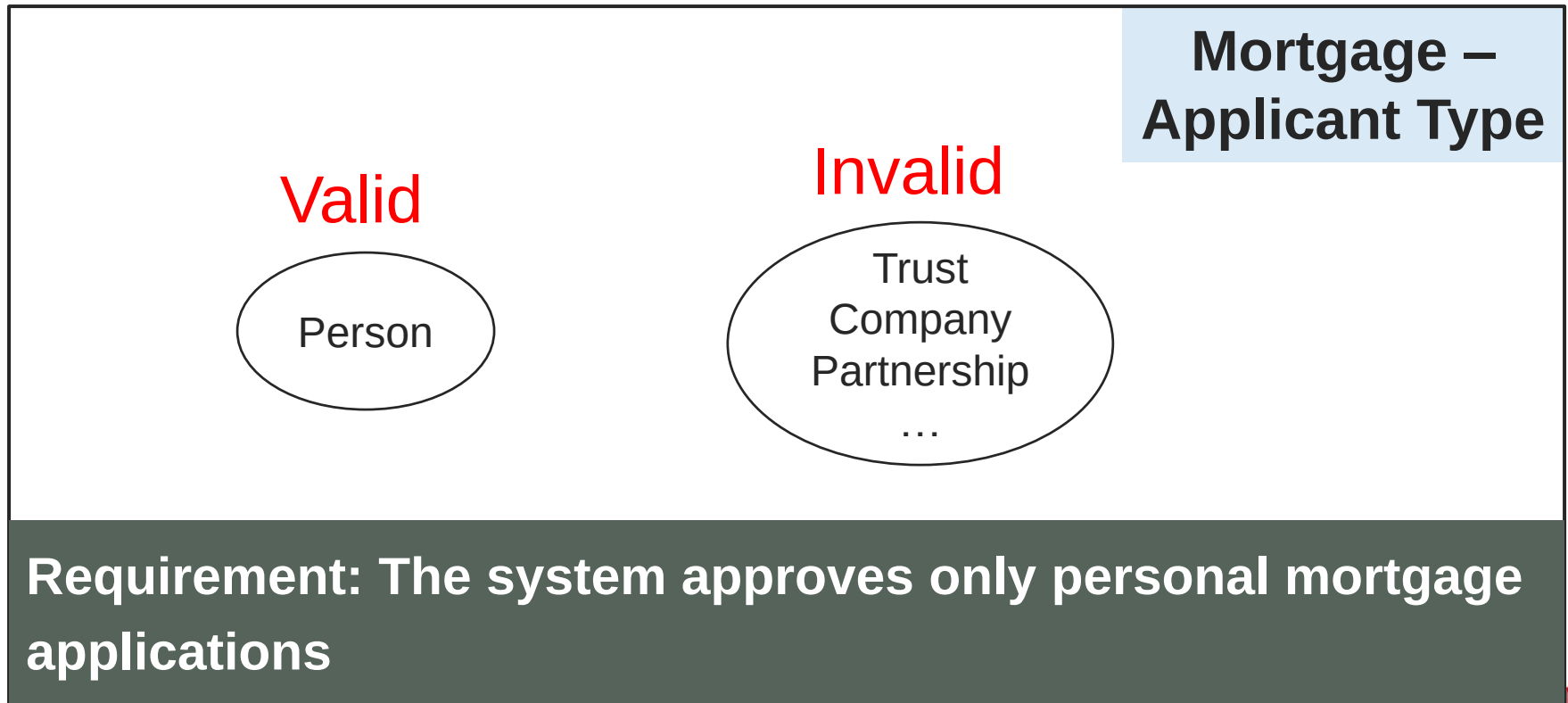
Invalid equivalence classes

$\text{age} \leq -1$
 $\text{age} \geq 100$

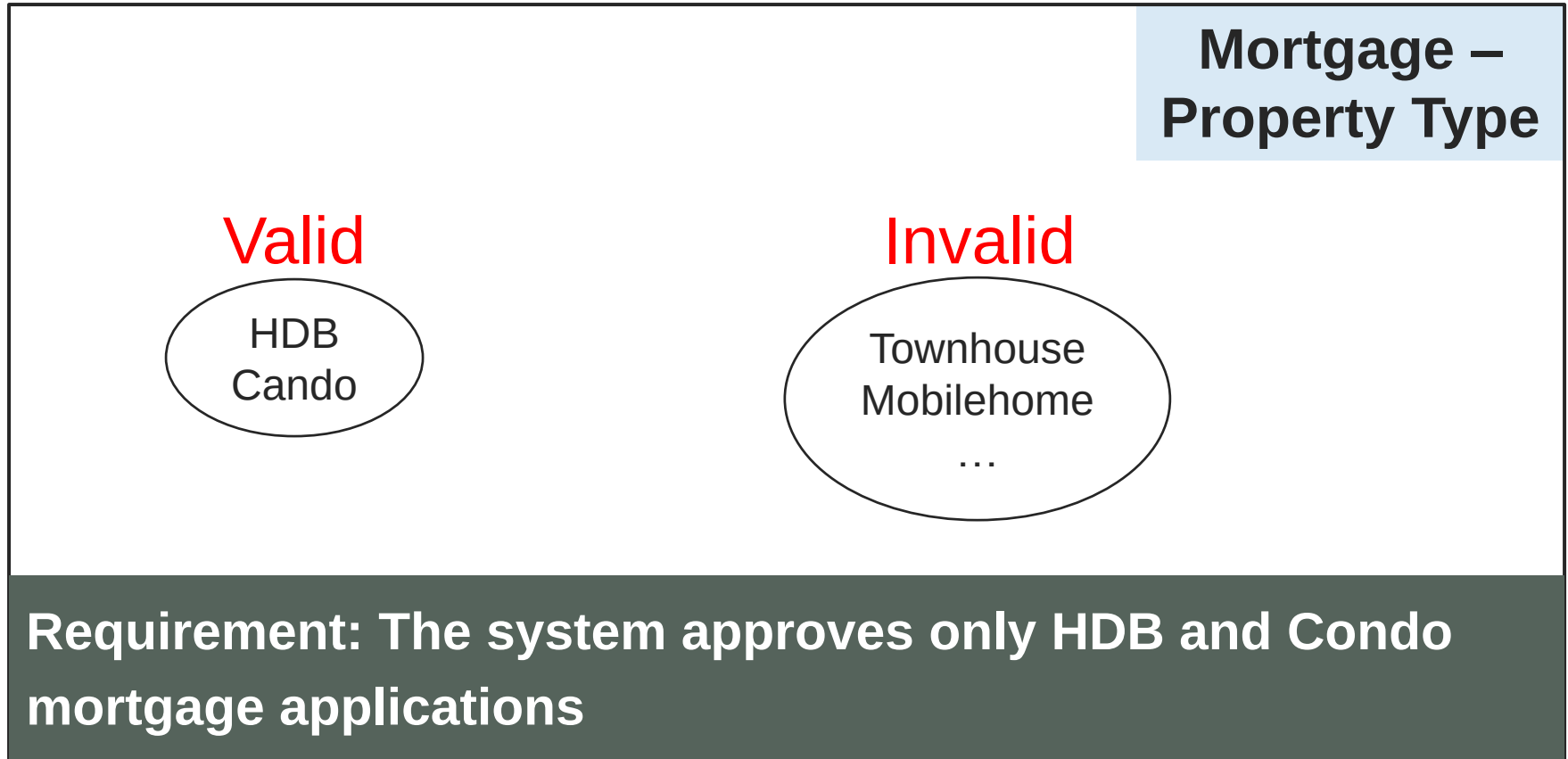
Requirements: 0 – 15, 55 – 99	Do not hire
16 – 17	Part-time
18 – 54	Full-time

Discrete Values

- ▶ **ONE** equivalence class of **VALID** values
- ▶ **ONE** equivalence class of **INVALID** values



Discrete Values – More Examples



Discrete Values – More Examples

Character of Password

Valid

a b ... z
A B ... Z
0 1 2 ... 9

Invalid

+ - () ...

Requirement: The password must be characters a-z, A-Z, and 0-9

Design Test Cases Using ECs

1. Test **VALID** inputs of several parameters
 - ▶ Test valid inputs of several parameters at the same time (i.e. per test case)
 - ▶ Choose one valid input for each input parameter from an equivalence class
2. Test **INVALID** inputs of several parameters
 - ▶ Test **ONLY ONE INVALID** input from an equivalence class of one input parameter at the same time (i.e. per test case)
 - ▶ Choose valid inputs from an equivalence class of all other input parameters

Test Valid Inputs Using ECs

- ▶ You may test **valid** input values of **several** input parameters at the same time
- ▶ You may create test cases for **all the values** in an valid equivalence class if the set of valid values in the equivalence class is **small**

e.g. The number of mortgages (0, 1, 2, 3, 4, 5)
Property type (HDB, Condo)

but not for income range, or characters of password

Test Valid Inputs – Example

Each row is a test case.
For each test case, choose one **valid** input value from a valid equivalence class of each parameter

As each parameter has only one valid EC, this single test case satisfies **EC testing principle**

Income	Number of mortgage	Applicant Type	Property Type	Expected Result
\$1500	0	Person	Condo	Approval
\$2000	1	Person	Condo	Approval
\$5000	2	Person	Condo	Approval
\$12345	3	Person	HDB	Approval
\$76543	4	Person	HDB	Approval
\$80000	5	Person	HDB	Approval

Try to cover a few more valid income value when enumerating #mortgage and PropertyType values

As #mortgage and PropertyType have **only 6 (0-5) and 2 (Condo, HDB)** valid values, we create several more test cases to cover **all** these values

Should We Test Invalid Inputs?

▶ Testing by Contract

- ▶ Create test cases **only** for **valid** inputs (i.e., the input values that satisfy the **pre-conditions**)

▶ Defensive Testing

- ▶ Create test cases for **BOTH VALID** and **INVALID** inputs
 - ▶ Do not assume that users will use your code the way it is supposed to be used
 - ▶ Test the module to see **whether it behaves as expected for invalid inputs**, i.e. do not cause unforeseen errors or abnormal system behavior (sudden program termination)

Which one does the testers prefer?

- ▶ If it does not work, who will get the blame? – the testers
- ▶ Testers prefer defensive testing

Test Invalid Inputs Using ECs

Why?

- ▶ You must test **ONLY ONE INVALID** input value from an equivalence class of one input parameter per test case i.e. each test case should have no more than one invalid input

Income	Number of mortgage	Applicant Type	Property Type	Expected Result
\$100	8	Company	Mobilehome	Reject

Invalid input Invalid input Invalid input Invalid input

If we test only one invalid input for each test case, then we can learn the error handling or behavior for each type of invalid input

Test Invalid Inputs Using ECs

- ▶ You may create test cases for **all the values** in an invalid equivalence class if the set of invalid values in the equivalence class is **small**

Test Invalid Inputs – Example

Each row is a test case.
Test **ONLY ONE INVALID**
input value **per test case**

Income	Number of mortgage	Applicant Type	Property Type	Expected Result
\$100	1	Person	Condo	Reject
\$81000	2	Person	HDB	Reject
\$5000	-1	Person	Condo	Reject
\$10000	8	Person	HDB	Reject
\$12345	3	Company	HDB	Reject
\$76543	4	Person	Mobilehome	Reject

Choose valid input values
for all other parameters

Quick Summary

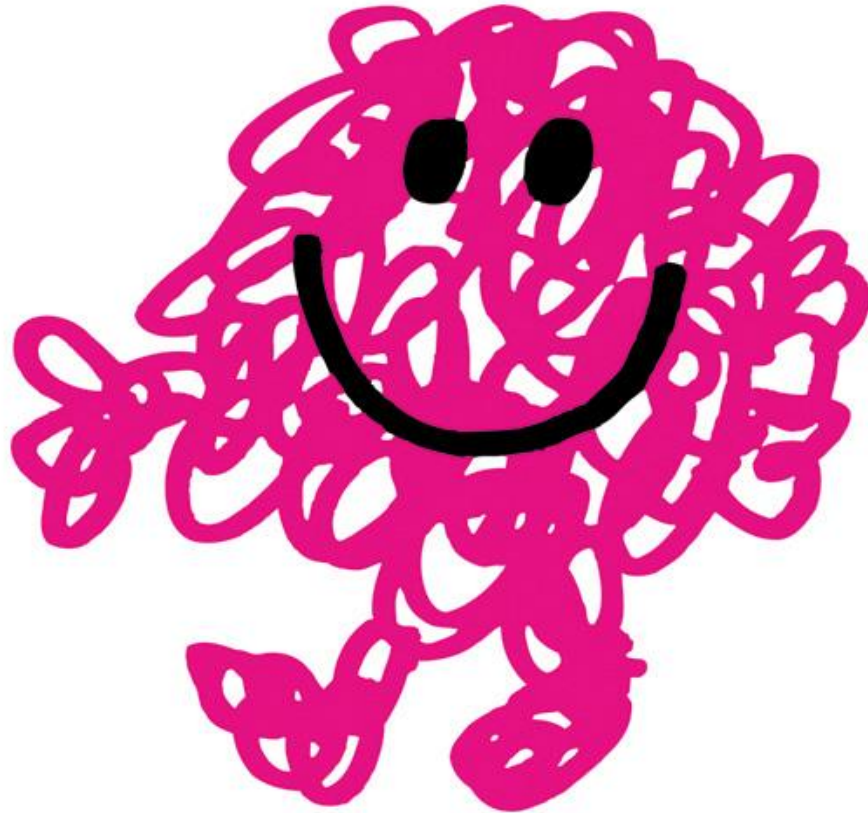
A key aspect of black box testing is identifying Equivalent Classes (EC).

Identifying ECs helps us test effectively with a minimal set of inputs.

For equivalence classes (ECs), range of values –

- **ONE VALID EC of VALID values**
- **TWO INVALID ECs of INVALID values**

But, the Real World is Messy



http://mrmen.wikia.com/wiki/File:MR_MESSY_4A.jpg

Real World Requirements are Messy Too 😊

Requirement ID	KBO_Req_01
Requirement Description	Administrator shall manually access the transaction information for a particular account on a periodic basis from the existing Master Ledger system. (Automation of this access mechanism is outside the scope of KBO's current release and may become a Requirement for a future release.) Administrator shall record the transaction information for a particular account accessed from the ML system, in the KBO system. Only successful transactions need to be available at KBO for viewing by users within 48 hours after they have been initiated by the user. Users will be notified of failed transactions through procedures outside the scope of KBO's current release.

Functional part	Non-functional part
Recording transaction details in KBO, by administrator	• Ensuring transactions are recorded in a format that makes it reasonably easy to be understood by users. • Ensuring transactions are correctly recorded for the appropriate user accounts. • Ensuring successful transactions are recorded at KBO within 48 hours of their initiation. • Ensuring large volumes of transaction for a particular account within a particular period of time can be handled.

A Key Skill is to Derive Verifiable Requirements for Effective Black Box Testing

Requirement ID	KBO_Req_01
Requirement Description	Administrator shall manually access the transaction information for a particular account on a periodic basis from the existing Master Ledger system. (Automation of this access mechanism is outside the scope of KBO's current release and may become a Requirement for a future release.) Administrator shall record the transaction information for a particular account accessed from the ML system, in the KBO system. Only successful transactions need to be available at KBO for viewing by users within 48 hours after they have been initiated by the user. Users will be notified of failed transactions through procedures outside the scope of KBO's current release.

Functional part	Non-functional part
Recording transaction details in KBO, by administrator	• Ensuring transactions are recorded in a format that makes it reasonably easy to be understood by users. • Ensuring transactions are correctly recorded for the appropriate user accounts. • Ensuring successful transactions are recorded at KBO within 48 hours of their initiation. • Ensuring large volumes of transaction for a particular account within a particular period of time can be handled.

Assumption #1

- ▶ **Verifiable requirements** and API specification exists
 - ▶ Make software more testable from its inception!